SimpleGrid problems compiled to d-DNNF with d4 (projected, no evidence): means by grid size

Grid	Problems	Horizon	d4 time (s)	Total time (s)	CNF variables	CNF clauses	Projected variables	Circuit nodes	Circuit edges	Model count	Query with 2 obs. (s)
3×3	4	5	0.04	2.0	141	818	98	44	84	18	0.001
4×4	4	7	0.04	1.9	400	2,609	291	228	451	140	0.003
5×5	4	8.5	0.09	2.0	812	5,621	604	1,108	2,210	1,017	0.006
6×6	4	10.75	1.36	4.0	1,557	11,191	1,175	21,140	42,296	24,171	0.017
7×7	4	12	6.78	11.7	2,436	17,936	1,854	82,748	165,584	88,126	0.036
8×8	4	14.75	161.71	–	4,024	30,205	3,087	785,660	1,573,583	5,431,088	–

Each row is the arithmetic mean over that grid size's problems (Problems gives how many), rounded; the per-problem values are in compile-d4.md. Each problem's evidence-free recognition theory — the problem at its recognition horizon, with the goal literals as the hypotheses, as recognize.sh builds it — compiled by marginals.sh `--solver` d4 `--project`. The projection keeps the drive actions and the final-slice goal atoms, and FiFO first verifies that they determine every other atom, so the projected circuit gives exact counts; any pinned observations are unit literals over the kept actions, so one saved circuit answers every evidence file by conditioning, without recompiling. CNF variables and clauses are those of the ground theory d4 is given; projected variables are the ones it keeps. d4 time is the d4 run alone; total time is the whole marginals.sh run (reading the theory, the projection check, d4, importing and saving the circuit, and the marginals of the unconditioned circuit). Circuit nodes and edges are those of d4's decision-DNNF, before FiFO smooths it. Times are wall-clock, 1 compilation at a time, except grid8x8-1 (2 at a time), each limited to 30 minutes. SBCL ran with an 8 GB heap and a 1 GB control stack, d4 with a 64 MB stack (the defaults, 1 GB, 2 MB and 8 MB, are too small for the larger circuits). The circuits were imported by FiFO's d4 importer as rewritten to use bit-vector scopes and iterative traversal (FiFO fd0d0b9); the earlier importer ran out of memory on circuits of about a million nodes. Model count (verified) is the number of models of the compiled circuit (all literals set to 1 in the smoothed circuit), that is, the number of truck walks the theory allows. Every circuit kept was checked against an independent count of those walks by dynamic programming over the grid (H-1 steps, each a drive to a neighbouring cell or a wait, from p1-1 to a goal cell), which agrees with sharpSAT on every problem up to 7×7 (sharpSAT did not finish an 8×8 problem in 2.5 hours); the two counts were equal for every circuit in the table. Grids past 7×7 were not re-run with the current FiFO: from 8×8 on, d4 often crashes (bus error or segmentation fault) or returns a wrong circuit without any error, and repeated runs on the same CNF give different circuits. Their rows give the run whose circuit passed the model-count check (grid8x8-1: correct on 1 of 1; grid8x8-2: correct on 1 of 8; grid8x8-3: correct on 1 of 10; grid8x8-4: correct on 1 of 6 runs); its d4 time is that run's alone, not counting failed runs. Their total time is not shown: it was measured with the earlier, quadratic evaluator (and partly two compilations at a time); and no queries were timed on them. Query with 2 obs. is the time to recompute every marginal of the saved circuit conditioned on two observations — the first two drives of a shortest plan to the problem's alphabetically first destination, pinned to steps 1 and 2 (evidence/PROBLEM-DESTINATION-obs2.txt) — as marginals.sh `--circuit --evidence-file` does: the observations are unit literals over kept atoms, so the circuit is clamped and re-evaluated, not recompiled. It is timed inside SBCL, the median of 5 repetitions, and excludes starting SBCL and loading FiFO (about 1.2 s) and reading the circuit file (0.001–0.45 s). Each answer was checked: both observed drives have probability 1, and the final-slice goal marginals sum to 1. FiFO evaluates the circuit in two linear passes; its backward pass through an AND node of k children uses prefix and suffix products (an earlier version recomputed each child's k-1 sibling products, k^2 multiplications, and took up to 7 s on 7×7 — smoothed AND nodes here have up to 1,007 children). In the run reported, d4 was given extra options: "`--cache-store-strategy` all `--cache-size-first-page` 268435456 `--cache-size-additional-page` 268435456" for grid8x8-2; "`--cache-store-strategy` all" for grid8x8-3 (every other problem: d4's defaults). Storing every clause in a cache entry, and cache pages smaller than the default 4 GB, made d4 miscompile less often, though not never. No 9×9 problem compiled within the limit, so larger grids were not tried.